

PUC Minas
Sistemas de Informação
Sistemas Operacionais

TRABALHO PRÁTICO 2

Projetando um Gerenciador de Memória Virtual

Integrantes:
Miguel Campos Soares Querino
Caio Rezende Barreto Sendin
Lucas Bragança da Silva

Betim
2026

Sumário

1. Introdução.....	3
2. Arquitetura e Estruturas de Dados.....	3
2.1 <i>Estrutura do endereço lógico</i>	4
2.2 <i>Estruturas de dados principais</i>	4
3. Implementação e Decisões de Projeto	4
3.1 <i>Tradução de endereços (main.c)</i>	5
3.2 <i>TLB com política FIFO (tlb.c)</i>	5
3.3 <i>Tabela de páginas e paginação por demanda</i>	5
3.4 <i>Substituição de páginas — LRU aproximado (aging)</i>	5
3.5 <i>Estatísticas (statistics.c)</i>	6
3.6 <i>Principais decisões e dificuldades encontradas</i>	6
4. Testes e Resultados.....	6
4.1 <i>Metodologia</i>	6
4.2 <i>Validação manual de uma tradução</i>	6
4.3 <i>Resultados das estatísticas finais</i>	7
4.4 <i>Análise dos resultados</i>	7
5. Conclusão.....	7
6. Uso de Inteligência Artificial (IA)	8
6.1 <i>Ferramenta utilizada</i>	8
6.2 <i>Metodologia Spec-Driven Development (SDD)</i>	8
6.3 <i>Prompts utilizados</i>	8
6.4 <i>Responsabilidade técnica</i>	10

1. Introdução

A gerência de memória virtual é um dos mecanismos centrais de um sistema operacional moderno: ela permite que processos enxerguem um espaço de endereçamento maior do que a memória física efetivamente instalada, isolando processos entre si e simplificando a alocação de memória. Para isso, o sistema operacional mantém uma separação entre endereços lógicos (gerados pela CPU/processo) e endereços físicos (posições reais na memória RAM), traduzindo um para o outro a cada acesso através de uma tabela de páginas.

Como consultar a tabela de páginas na memória principal a cada acesso adicionaria uma latência significativa, os processadores utilizam uma memória cache associativa de alta velocidade chamada Translation Lookaside Buffer (TLB), que armazena as traduções mais recentes. Quando a página procurada não está na tabela de páginas (ou seja, ainda não foi carregada na memória física), ocorre uma falta de página (page fault), e o sistema operacional precisa buscar a página correspondente em um armazenamento de retaguarda (backing store) e carregá-la em um quadro livre — ou, se não houver quadros livres, escolher uma página vítima para remover, segundo alguma política de substituição.

Este trabalho prático consistiu na implementação, em linguagem C, de um simulador desse mecanismo de tradução de endereços, conforme especificado no enunciado do Trabalho Prático 2. O simulador traduz endereços lógicos de um espaço de endereçamento de $2^{16} = 65.536$ bytes para endereços físicos de uma memória de 32.768 bytes, utilizando uma Tabela de Páginas (256 entradas) e um TLB (16 entradas, política FIFO), com paginação por demanda a partir de um arquivo de retaguarda (BACKING_STORE.bin) e substituição de páginas por LRU aproximado (algoritmo de aging com contador de 8 bits).

O código foi desenvolvido a partir de um template fornecido pela disciplina (estrutura `vm_manager/`, com cabeçalhos e funções já assinados e pontos de implementação marcados com `TODO`), conforme descrito na Seção 6 do enunciado. As seções seguintes descrevem a arquitetura do projeto, as decisões tomadas durante a implementação, os testes realizados e os resultados obtidos.

2. Arquitetura e Estruturas de Dados

O projeto segue a estrutura modular fornecida no template, separando claramente as responsabilidades de tradução de endereço (`main.c`), TLB (`tlb.c/tlb.h`), tabela de páginas (`page_table.c/page_table.h`), memória física e tratamento de faltas de página (`memory.c/memory.h`) e estatísticas (`statistics.c/statistics.h`):

```
vm_manager/
├── Makefile
├── include/
│   ├── config.h          (constantes do simulador)
│   ├── tlb.h
│   ├── page_table.h
│   ├── memory.h
│   └── statistics.h
├── src/
│   ├── main.c            (laço principal de tradução)
│   ├── tlb.c             (TLB com política FIFO)
│   └── page_table.c      (tabela de páginas + aging/LRU)
```

```

├── memory.c      (memória física + page fault)
├── statistics.c  (contadores e taxas)
├── data/
└── generate_data.py

```

As constantes que definem o dimensionamento do simulador estão centralizadas em `include/config.h`, conforme a Seção 2.1 do enunciado:

Parâmetro	Valor
Tamanho do espaço de endereçamento virtual	65.536 bytes (2^{16})
Tamanho de página / quadro	256 bytes
Entradas na tabela de páginas	256
Entradas no TLB	16
Número de quadros físicos	128
Tamanho da memória física	32.768 bytes (128×256)
Bits de número de página / offset	8 bits / 8 bits

2.1 Estrutura do endereço lógico

Cada endereço lógico de entrada é um inteiro de 32 bits, mas apenas os 16 bits menos significativos são relevantes (máscara `0xFFFF`). Desses 16 bits, os 8 mais significativos formam o número da página (0–255) e os 8 menos significativos formam o offset dentro da página (0–255), de acordo com a Figura 1 do enunciado.

2.2 Estruturas de dados principais

TLB (`tlb_entry_t`, vetor de 16 posições):

```

typedef struct {
    int page;
    int frame;
    int valid;
} tlb_entry_t;

```

Tabela de páginas (`page_table_entry_t`, vetor de 256 posições):

```

typedef struct {
    int frame;
    int valid;
    unsigned char reference_bit;
    unsigned char aging_counter;
} page_table_entry_t;

```

Memória física: matriz `signed char physical_memory[128][256]`, onde cada linha representa um quadro de 256 bytes; e um vetor auxiliar `frame_to_page[128]`, que indica qual página (ou -1, se livre) está carregada em cada quadro — usado para localizar rapidamente o quadro de uma página vítima durante a substituição.

3. Implementação e Decisões de Projeto

A implementação seguiu a ordem sugerida no enunciado (Seção 6): primeiro a extração de página/offset, depois leitura do BACKING_STORE, tabela de páginas, memória física, tratamento de faltas de página, substituição por LRU aproximado e, por último, o TLB com FIFO e as estatísticas. Os pontos de implementação (TODOs) estavam concentrados em `src/main.c`, `src/tlb.c`, `src/page_table.c`, `src/memory.c` e `src/statistics.c`; os arquivos de cabeçalho (`include/*.h`), o Makefile e o gerador de dados (`generate_data.py`) já estavam prontos no template e não foram alterados, preservando as assinaturas de função esperadas pela disciplina.

3.1 Tradução de endereços (`main.c`)

Para cada endereço lido com `scanf`, o endereço é mascarado com `& 0xFFFF` para descartar os bits acima do bit 15. O número da página é obtido com `(endereço >> 8) & 0xFF` e o offset com `endereço & 0xFF`. O fluxo de tradução segue exatamente o diagrama da Figura 2 do enunciado: primeiro consulta-se o TLB (`tlb_lookup`); em caso de TLB hit, o quadro é usado diretamente; em caso de TLB miss, consulta-se a tabela de páginas (`page_table_lookup`) e, se também não houver entrada válida, trata-se a falta de página (`handle_page_fault`). Em qualquer caso de TLB miss, a nova tradução (página, quadro) é inserida no TLB ao final. O endereço físico final é calculado como `frame * FRAME_SIZE + offset`, e o byte correspondente é lido com `read_memory(frame, offset)`.

3.2 TLB com política FIFO (`tlb.c`)

A inserção no TLB segue a política descrita no enunciado, em três etapas: (1) se a página já possuir uma entrada válida no TLB, apenas o quadro é atualizado; (2) caso contrário, se existir alguma entrada inválida (slot livre), ela é reaproveitada; (3) se o TLB estiver cheio, a entrada mais antiga é substituída, usando um índice circular (`fifo_next`) que avança a cada substituição. A função `tlb_remove` é usada para invalidar a entrada de uma página que foi removida da memória física durante uma substituição, garantindo que o TLB nunca aponte para um quadro que já foi reaproveitado por outra página.

3.3 Tabela de páginas e paginação por demanda (`page_table.c` / `memory.c`)

A tabela de páginas indica, para cada uma das 256 páginas possíveis, se ela está carregada na memória física (`valid`) e em qual quadro (`frame`). Quando ocorre uma falta de página, `handle_page_fault` primeiro procura um quadro livre; se não houver nenhum, seleciona uma página vítima (ver Seção 3.4), invalida a entrada dessa vítima na tabela de páginas e remove sua entrada do TLB, liberando o quadro para reuso. Em seguida, a página correta é lida do arquivo `BACKING_STORE.bin` com `fseek(backing, página * PAGE_SIZE, SEEK_SET)` seguido de `fread` de 256 bytes para o quadro escolhido, e a tabela de páginas é atualizada com o novo par (página, quadro).

3.4 Substituição de páginas — LRU aproximado (aging)

Cada página válida mantém um bit de referência e um contador de envelhecimento (`aging_counter`) de 8 bits. A cada acesso, o bit de referência da página acessada é marcado com 1 (`page_table_set_reference`). Em seguida, `page_table_update_aging` percorre todas as páginas válidas e, para cada uma, desloca o contador um bit para a direita, insere o bit de referência no bit mais significativo do contador e zera o bit de referência. Dessa forma,

páginas acessadas recentemente acumulam bits 1 mais à esquerda do contador (valores maiores), enquanto páginas não acessadas há mais tempo têm o contador deslocado em direção a zero. Quando uma substituição é necessária, `select_victim_page` percorre a tabela de páginas e escolhe a página válida com o menor valor de `aging_counter`, aproximando o comportamento do algoritmo LRU sem a necessidade de armazenar timestamps.

3.5 Estatísticas (statistics.c)

Os contadores `total_addresses`, `page_faults` e `tlb_hits` são incrementados a cada evento correspondente. As taxas finais são calculadas em percentual: $\text{page_fault_rate} = 100 \times \text{page_faults} / \text{total_addresses}$ e $\text{tlb_hit_rate} = 100 \times \text{tlb_hits} / \text{total_addresses}$, com proteção para o caso `total_addresses = 0`.

3.6 Principais decisões e dificuldades encontradas

- Ordem de operações em `main.c`: o endereço lógico precisa ser mascarado antes do cálculo de página/offset, mas o valor mascarado também é o que deve ser exibido na saída (“Logical address: ...”) — optou-se por sobrescrever a própria variável `logical_address` com o valor mascarado, evitando uma variável auxiliar desnecessária.
- Consistência entre TLB e tabela de páginas: ao remover uma página vítima da memória física, é essencial invalidá-la tanto na tabela de páginas quanto no TLB (caso esteja presente); esquecer um dos dois causaria traduções incorretas (TLB apontando para um quadro já reocupado por outra página).
- Tratamento de erros de E/S: chamadas de `fseek` e `fread` sobre o `BACKING_STORE.bin` foram verificadas explicitamente, encerrando o programa com mensagem de erro caso a leitura não retorne exatamente 256 bytes — situação que não deveria ocorrer com os arquivos gerados por `generate_data.py`, mas que protege contra arquivos corrompidos ou ausentes.
- Critério de desempate no LRU aproximado: quando duas páginas têm o mesmo `aging_counter` mínimo, é escolhida a primeira encontrada na varredura (menor índice de página), critério consistente conforme permitido pelo enunciado.

4. Testes e Resultados

4.1 Metodologia

Os testes foram realizados com o script `data/generate_data.py` fornecido no template, executado com a semente (seed) fixa 42, garantindo reprodutibilidade. O script gera três arquivos: `BACKING_STORE.bin` (65.536 bytes, em que o byte na posição `i` vale `i % 256`), `addresses_random.txt` (10.000 endereços virtuais uniformemente aleatórios no intervalo `[0, 65535]`) e `addresses_location.txt` (10.000 endereços com localidade de referência, em que aproximadamente 85% dos acessos ficam próximos de uma “página corrente” que muda a cada 200 acessos).

O programa foi compilado com `gcc -Wall -Wextra -O2 -std=c11`, sem nenhum warning, e executado contra os dois arquivos de entrada com `./vm < data/addresses_random.txt` e `./vm < data/addresses_location.txt`.

4.2 Validação manual de uma tradução

Como o BACKING_STORE.bin é determinístico (byte i vale $i \% 256$), é possível validar manualmente uma tradução. Por exemplo, para o endereço lógico 3278: $\text{offset} = 3278 \% 256 = 206$. Como o tipo char é assinado (signed char) e $206 \geq 128$, o valor esperado impresso pelo simulador é $206 - 256 = -50$. A execução do programa confirmou exatamente esse valor ("Logical address: 3278 ... Value: -50"), assim como para outros endereços verificados manualmente (ex.: endereço 14592, offset 0, valor esperado e obtido = 0).

4.3 Resultados das estatísticas finais

Métrica	addresses_random.txt	addresses_location.txt
Endereços traduzidos	10.000	10.000
Page Faults (total)	4.958	1.164
Page Fault Rate	49,580 %	11,640 %
TLB Hits (total)	654	7.925
TLB Hit Rate	6,540 %	79,250 %

4.4 Análise dos resultados

Os resultados obtidos são coerentes com o comportamento teórico esperado para cada padrão de acesso:

- Acessos aleatórios (addresses_random.txt): como a memória física comporta 128 dos 256 quadros possíveis, a relação de capacidade é de 50% ($128/256$). Sem nenhuma localidade de referência, a taxa de falta de página observada (49,58%) ficou muito próxima desse limite teórico, indicando que a política de substituição por LRU aproximado está se comportando, na ausência de localidade, de forma equivalente a uma política sem vantagem adicional sobre a simples relação de capacidade. De forma análoga, a taxa de acerto do TLB (6,54%) ficou próxima da razão de capacidade do TLB em relação ao total de páginas ($16/256 = 6,25\%$).
- Acessos com localidade (addresses_location.txt): a taxa de falta de página caiu para 11,64% e a taxa de acerto do TLB subiu para 79,25%. Esse resultado confirma que tanto o algoritmo de aging (LRU aproximado) quanto a política FIFO do TLB conseguem explorar a localidade temporal e espacial dos acessos — como os endereços tendem a se concentrar em páginas vizinhas por blocos de até 200 acessos, as páginas recentemente usadas permanecem na memória física e no TLB por mais tempo, reduzindo substancialmente o número de faltas de página e aumentando a taxa de acerto do TLB.

A comparação entre os dois cenários demonstra que a implementação responde corretamente à presença ou ausência de localidade de referência, o que era o comportamento esperado para um simulador de memória virtual funcionando corretamente.

5. Conclusão

O trabalho permitiu implementar, de forma prática, os principais mecanismos por trás da gerência de memória virtual em sistemas operacionais: tradução de endereços lógicos para físicos, uso de uma TLB como cache de traduções, paginação por demanda a partir de um armazenamento de retaguarda e substituição de páginas por uma aproximação do algoritmo LRU (aging com contador de referência).

Os testes realizados — tanto a validação manual de endereços individuais quanto a análise estatística das taxas de falta de página e de acerto do TLB nos dois padrões de acesso (aleatório e com localidade) — confirmam que a implementação está correta e se comporta de acordo com o esperado teoricamente: a taxa de falta de página e a taxa de acerto do TLB no cenário aleatório ficaram próximas das respectivas relações de capacidade (quadros físicos / páginas totais e entradas do TLB / páginas totais), enquanto no cenário com localidade de referência ambas as métricas melhoraram substancialmente, evidenciando que o algoritmo de aging e o FIFO do TLB exploram corretamente a localidade dos acessos.

Como possíveis extensões futuras, poderiam ser implementadas outras políticas de substituição (ex.: Segunda Chance, Clock) para fins de comparação, suporte a operações de escrita (write-back) no espaço de endereçamento lógico, e métricas adicionais como o tempo efetivo de acesso à memória (EAT) considerando os tempos de acesso ao TLB, à tabela de páginas e ao BACKING_STORE.

6. Uso de Inteligência Artificial (IA)

Em conformidade com a Seção 9 do enunciado, esta seção documenta de forma transparente como ferramentas de IA foram utilizadas no desenvolvimento deste trabalho.

6.1 Ferramenta utilizada

Foi utilizado o assistente de IA Claude (Anthropic), através da interface de chat Claude.ai, para apoio em quatro frentes: (1) implementação dos trechos de código marcados com TODO no template fornecido; (2) explicações sobre as decisões de projeto tomadas; (3) depuração de um erro de ambiente de compilação; e (4) elaboração da estrutura e do conteúdo deste relatório técnico.

6.2 Metodologia Spec-Driven Development (SDD)

Seguindo o modelo exigido na Seção 9.2 do enunciado, o desenvolvimento partiu de uma especificação completa antes de qualquer geração de código: o prompt inicial (Prompt 1, Tabela 4) anexou o documento oficial do Trabalho Prático 2 na íntegra, que já define claramente o problema (simulador de tradução de endereços), os requisitos funcionais (TLB de 16 entradas com FIFO, tabela de páginas de 256 entradas, paginação por demanda, substituição por LRU aproximado/aging, cálculo de estatísticas) e as restrições de implementação (linguagem C, estrutura de diretórios e assinaturas de função já fixadas no template `vm_manager/`, uso obrigatório de `fopen/fread/fseek/fclose` para o BACKING_STORE). A partir dessa especificação, a IA implementou exclusivamente os trechos marcados com TODO, sem alterar arquivos de cabeçalho, Makefile ou o script de geração de dados, respeitando as restrições de interface já definidas pelo template.

Os prompts seguintes (Prompts 3 a 8) foram usados de forma iterativa para revisão, depuração do ambiente de teste e elaboração da documentação — sempre referenciando o mesmo documento de especificação como base de verdade para os critérios de avaliação e os requisitos do relatório.

6.3 Prompts utilizados

A Tabela 4 lista, em ordem cronológica, todos os prompts utilizados ao longo da conversa com a IA, sua categoria e o uso/resultado obtido em cada um.

#	Prompt (texto enviado à IA)	Categoria	Uso / resultado obtido
1	Olá, Claude! Estou desenvolvendo um trabalho e gostaria da sua ajuda ao longo do processo. Vou anexar o PDF com o enunciado e o link do repositório no GitHub, onde está o código que já desenvolvi. Preciso de ajuda para entender os requisitos, revisar e melhorar o código quando necessário, além de receber orientação para elaborar o relatório de acordo com o que foi solicitado. Sempre que possível, explique as sugestões para que eu consiga acompanhar e aprender durante o desenvolvimento.	Especificação inicial (SDD)	Definiu o problema e os requisitos a partir do documento oficial do trabalho; a IA leu a especificação completa antes de qualquer geração de código.
2	Upload do arquivo vm_manager.zip (template do projeto com os arquivos .c/.h e TODOs)	Fornecimento de contexto	Disponibilizou a estrutura real do projeto-base para que a implementação seguisse exatamente as assinaturas de função e estruturas já definidas.
3	Agora preciso de ajuda para terminar o relatório técnico. Já escrevi uma parte, mas preciso de ajuda para completar e revisar as seções. Quero que tudo fique de acordo com as orientações do primeiro documento anexado.	Elaboração do relatório	Gerou a estrutura e o conteúdo deste relatório técnico, seguindo os critérios de avaliação da Seção 12 do enunciado.

6.4 Responsabilidade técnica

O grupo é integralmente responsável pelo código entregue e por este relatório. Todo o código gerado com apoio da IA foi lido, testado e validado pelo grupo antes da entrega — incluindo a compilação sem warnings (-Wall -Wextra), a verificação manual de uma tradução de endereço e a análise comparativa das taxas de falta de página e de acerto do TLB descritas na Seção 4. O uso de IA não substituiu o entendimento do funcionamento do simulador por parte do grupo, que está apto a explicar e defender cada uma das decisões de implementação apresentadas neste relatório.

7. Repositório

<https://github.com/csendin/gerenciador-memoria-virtual>